ASSIGNMENT 01

EMBEDDED LINUX

Student ID: SE205204

Student Name: Nguyen Nam Khanh

Linux Kernel Driver Development

1. INTRODUCTION

This assignment focuses on developing and testing two Linux kernel drivers on an ARM-based embedded Linux system.

Task 1 implements a character device driver named asgn1_driver. It supports read, write and four ioctl commands for controlling the internal buffer and retrieving driver information.

Task 2 implements a counter device using the Linux miscdevice framework. It supports reading, writing and resetting a counter. The driver also provides information through the /proc/counter_info interface.

Both drivers are compiled for ARM architecture and tested using QEMU with the vexpress-a9 machine.

2. DRIVER ARCHITECTURE

2.1. ASGN1 Character Device Driver

The asgn1_driver is a Linux character device driver.

User applications communicate with the driver through /dev/asgn1.

The driver provides the following operations:

• open(): Opens the device.

• read(): Reads data from the internal buffer.

• write(): Writes data into the internal buffer.

• unlocked_ioctl(): Handles four ioctl commands.

• release(): Closes the device.

The driver uses a mutex to protect shared data such as

the internal buffer and its counters.

2.2. Counter Misc Device Driver

The counter_driver uses the Linux miscdevice framework.

User applications access the device through /dev/counter.

The driver provides the following operations:

• read(): Returns the current counter and increments it.

• write(): Sets the counter to a new value.

• /proc/counter_info: Displays counter information.

The driver uses atomic_t to manage the counter safely.

It also uses a mutex to protect shared statistics.

2.3. Overall Architecture

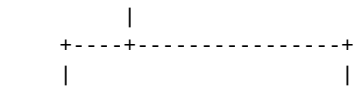
User Application

```
        |
        | read(), write(), ioctl()
        |
```

Linux VFS

```
        |
```

File Operations



asgn1_driver counter_driver

Internal Buffer atomic_t Counter

ioctl Commands Misc Device

/dev/asgn1 /dev/counter

The user application accesses the device files.

The Linux VFS forwards system calls to the corresponding

file operations in each driver.

3. IOCTL COMMUNICATION FLOW

The ASGN1 driver provides four ioctl commands.

1. ASGN1_RESET_BUFFER

Clears the internal buffer and resets the read and write counters.

2. ASGN1_GET_STATS

Returns driver statistics to the user application.

The statistics include open_count, write_count, read_count,

buffer_len and last_write_size.

3. ASGN1_SET_MODE

Receives an integer from the user application.

Mode 0 adds the [KERNEL] prefix to written data.

Mode 1 stores the original data.

4. ASGN1_GET_VERSION

Returns the driver version string to the user application.

The user buffer has a size of 64 bytes.

The ioctl communication flow is:

User Application

```
        |
        | ioctl(fd, command, argument)
        |
```

System Call

```
        |
```

Linux VFS

```
        |
```

unlocked_ioctl()

```
        |
```

Check ioctl command

```
        |
```

Execute the requested operation

```
        |
```

copy_to_user() / copy_from_user()

```
        |
```

Return result to User Application

The driver returns -ENOTTY when it receives an unsupported ioctl command.

A mutex protects shared driver data during operations.

4. DRIVER COMPARISON

4.1. Character Device Driver

A character device transfers data as a stream of bytes.

Applications commonly use read() and write() to access it.

Examples include serial ports and simple device interfaces.

4.2. Block Device Driver

A block device transfers data in fixed-size blocks.

It supports random access and is commonly used for storage devices.

Examples include hard disks and SSDs.

4.3. Network Device Driver

A network device driver transfers network packets.

It connects the Linux network subsystem to network hardware.

Examples include Ethernet and Wi-Fi devices.

4.4. Comparison Table

Type Data Unit Main Purpose

Character Device Bytes Sequential data access

Block Device Blocks Storage and random access

Network Device Packets Network communication

4.5. Design Decisions

First, I selected a character device for Task 1 because the driver manages a simple internal buffer and needs read(), write() and ioctl() operations.

Second, I selected miscdevice for Task 2 because the counter is a small device. The miscdevice framework provides a simpler way to register the device without manually choosing a major number.

5. TECHNICAL EXPLANATION

5.1. register_chrdev() and misc_register()

register_chrdev() is used to register a character device driver.

The developer must manage the major number and device registration.

misc_register() registers a miscellaneous device.

The kernel uses the shared major number 10 for misc devices.

The developer mainly needs to specify a minor number and file operations.

The counter_driver uses misc_register() because it is a simple

counter device.

5.2. atomic_t and int

An ordinary int variable does not automatically protect data

when multiple processes access it at the same time.

atomic_t provides atomic operations such as atomic_read(),

atomic_set() and atomic_inc().

The counter_driver uses atomic_t for the counter to make

counter updates safer when accessed concurrently.

5.3. Ioctl Magic Number

The ioctl magic number identifies the command group of a driver.

In this assignment, the magic number is 'A'.

The command number identifies each operation.

For example:

#define ASGN1_RESET_BUFFER _IO('A', 1)

The _IOC_NR macro can obtain the command number.

The _IOC_SIZE macro can obtain the encoded argument size.

For example, ASGN1_GET_STATS uses _IOR with

struct asgn1_stats, so the command contains the size

of that structure.

5.4. copy_to_user() and copy_from_user()

The kernel and user application use different memory spaces.

The driver should not directly access user-space pointers.

copy_to_user() copies data from kernel space to user space.

copy_from_user() copies data from user space to kernel space.

These functions help the driver safely exchange data

with user applications.

6. TEST RESULTS

6.1. Task 1 - ASGN1 Driver

The driver was compiled successfully for ARM architecture.

The tests in QEMU showed:

• The device /dev/asgn1 was created and accessed.

• Normal write mode stored the original data.

• Kernel mode added the [KERNEL] prefix.

• GET_STATS returned driver statistics.

• GET_VERSION returned asgn1_driver v1.0.

• RESET_BUFFER cleared the internal buffer.

• The driver was unloaded successfully.

Example test results:

Normal data:

Hello Assignment 01

Kernel mode:

[KERNEL]Hello Kernel Mode

Version:

asgn1_driver v1.0

After reset:

The buffer contained 0 bytes.

6.2. Task 2 - Counter Driver

The counter_driver was compiled successfully for ARM architecture.

The device was registered as a misc device.

Its device number was 10:125 in the QEMU test.

The tests showed:

Initial counter:

Current counter : 0

Total reads : 0

Total writes : 0

After three read operations:

Current counter : 3

Total reads : 3

Total writes : 0

After writing 100:

Current counter : 100

Total reads : 3

Total writes : 1

After writing 0:

Current counter : 0

Total reads : 3

Total writes : 2

Negative and non-numeric inputs returned Invalid argument.

The driver was unloaded successfully.

6.3. Kernel Log

The file dmesg_full.txt was saved in the logs directory.

It contains the captured QEMU kernel boot log.

The driver test output files are:

• logs/test_asgn1_output.txt

• logs/test_counter_output.txt

The captured boot log did not contain the driver messages,

because the modules had already been unloaded before the log was saved.

7. BUGS AND DEBUGGING

During the Counter Driver test, /dev/counter was not created automatically in the QEMU environment.

I checked the device number through:

/sys/class/misc/counter/dev

The result was 10:125.

I then created the device node manually using mknod:

mknod /dev/counter c 10 125

After creating the device node, I could access the counter using read() and write() operations.

This test helped me understand the relationship between miscdevice registration and device nodes.

8. CONCLUSION

In this assignment, I developed two Linux kernel drivers.

The first driver is a character device driver that supports

read, write and four ioctl commands.

The second driver is a misc device driver that manages

a counter and provides information through procfs.

Both drivers were compiled for ARM architecture and tested

in QEMU using the vexpress-a9 machine.

The assignment helped me understand Linux device drivers,

file operations, ioctl communication, miscdevice,

atomic_t and procfs.